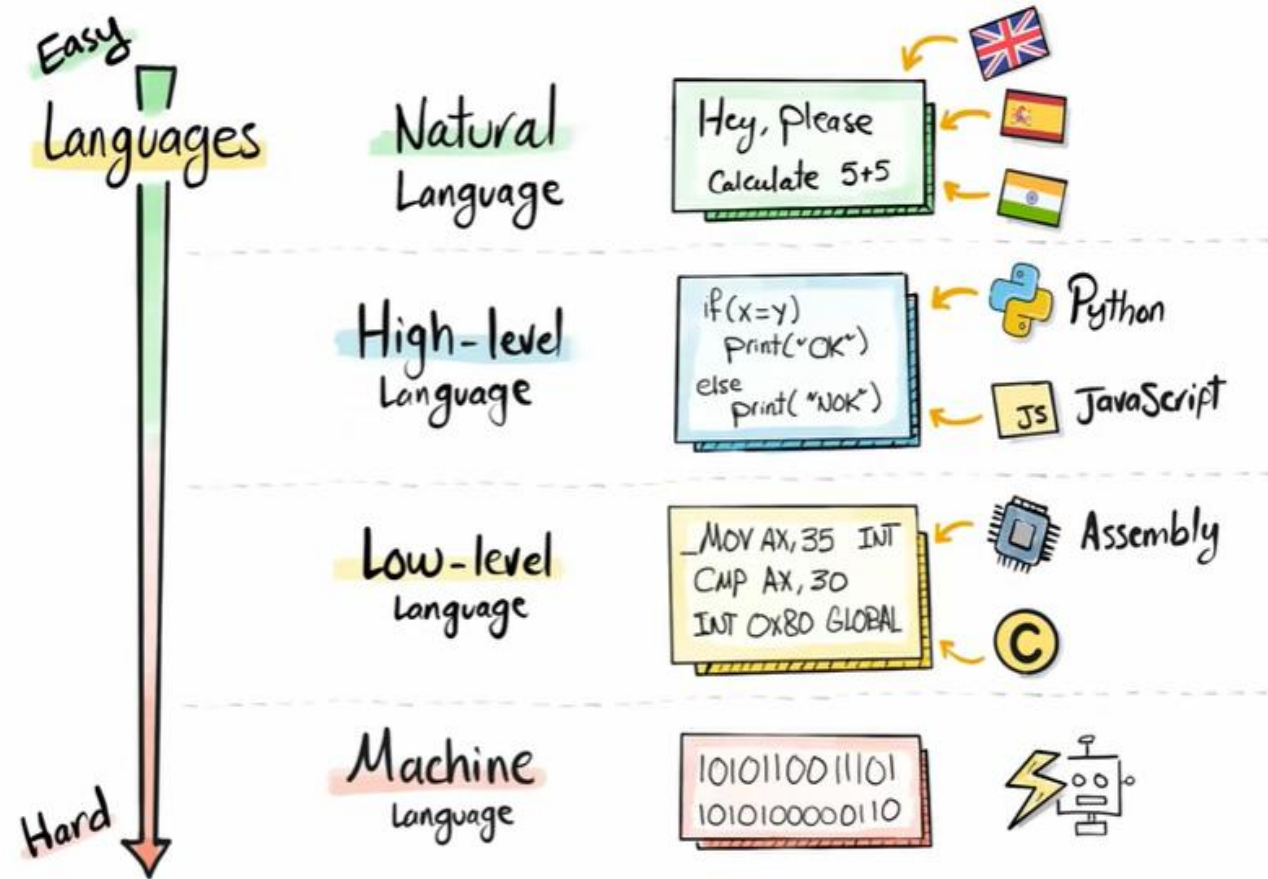
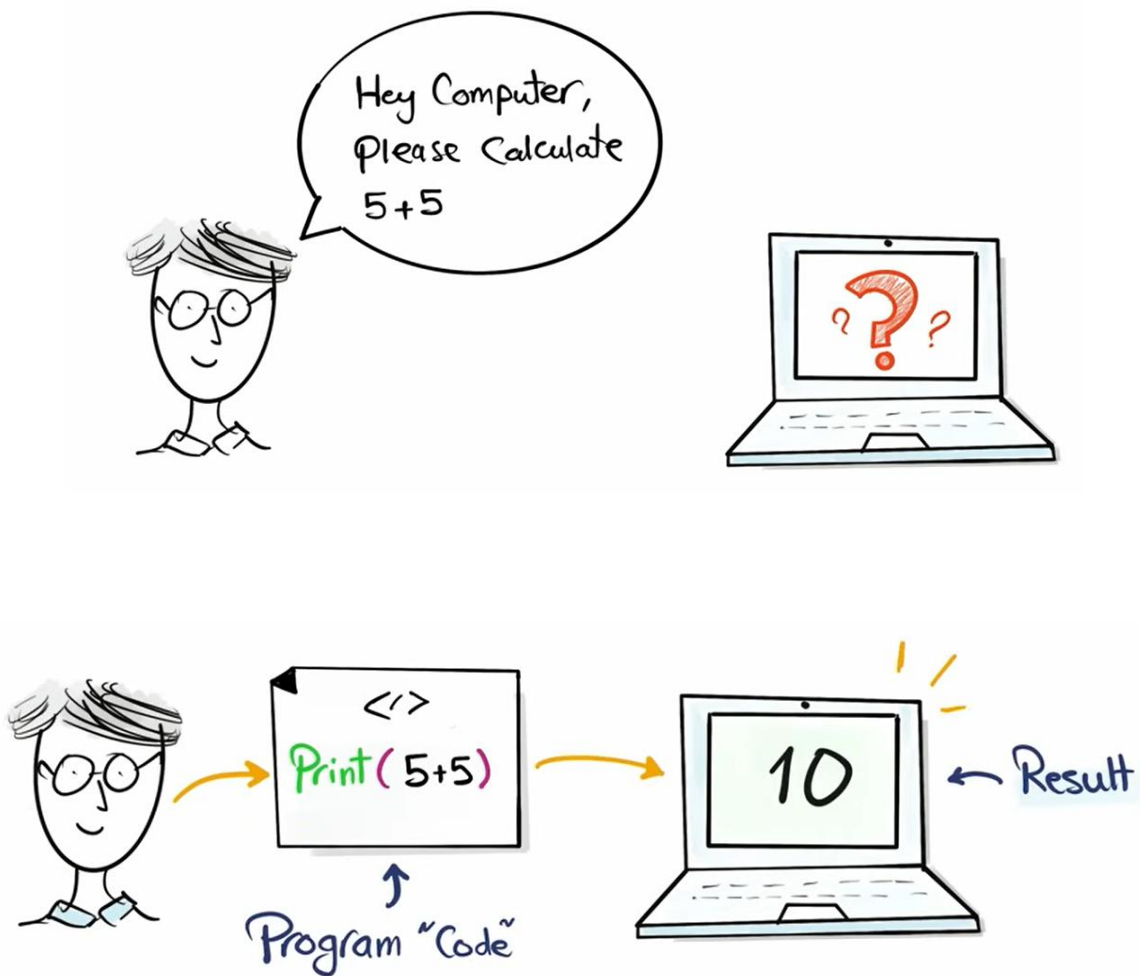
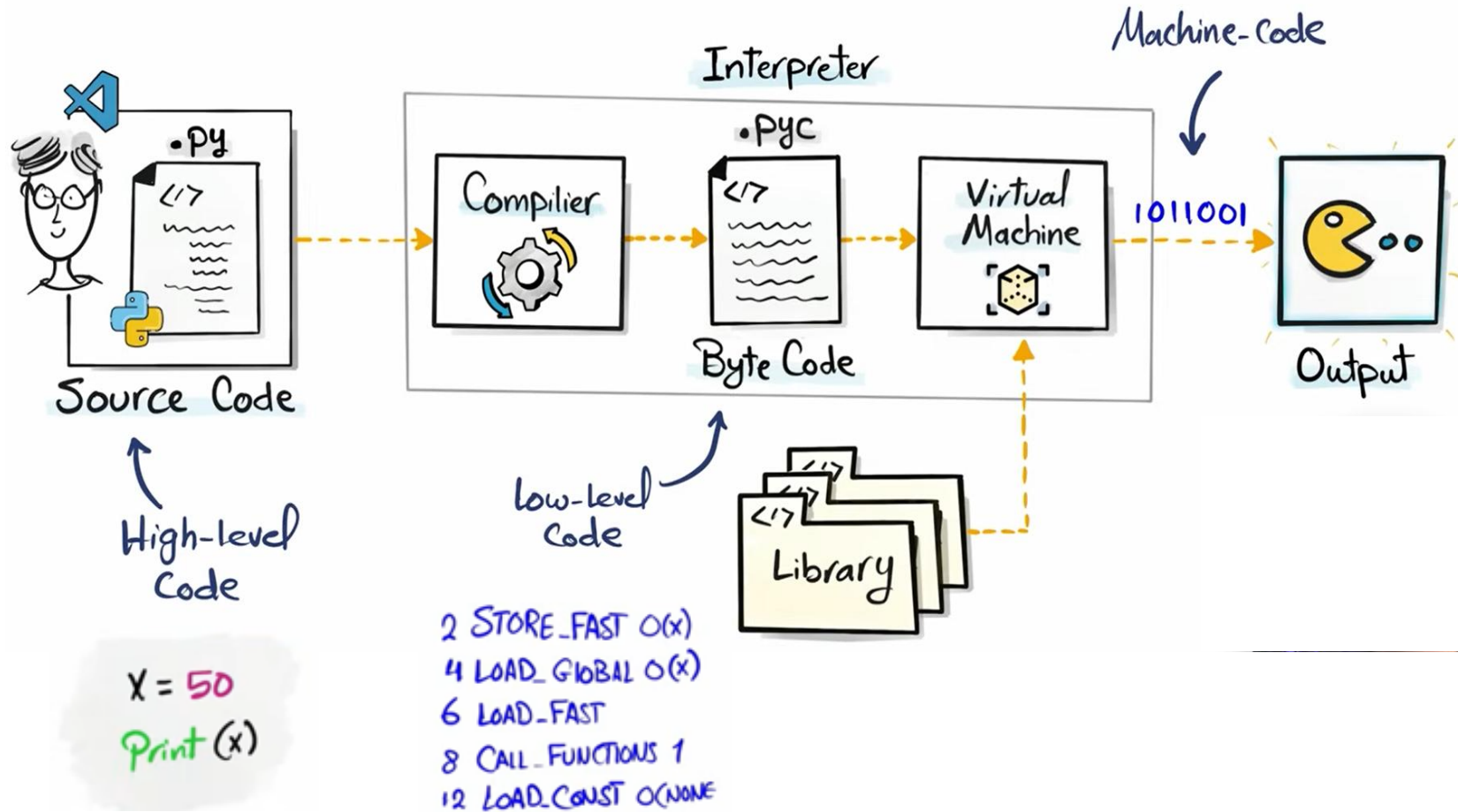


PYTHON

INTRODUCTION



How Python Works?



Python Code .py

```
1 a = 5
2 b = 10
3 sum = a + b
4 print("The sum is:", sum)
```

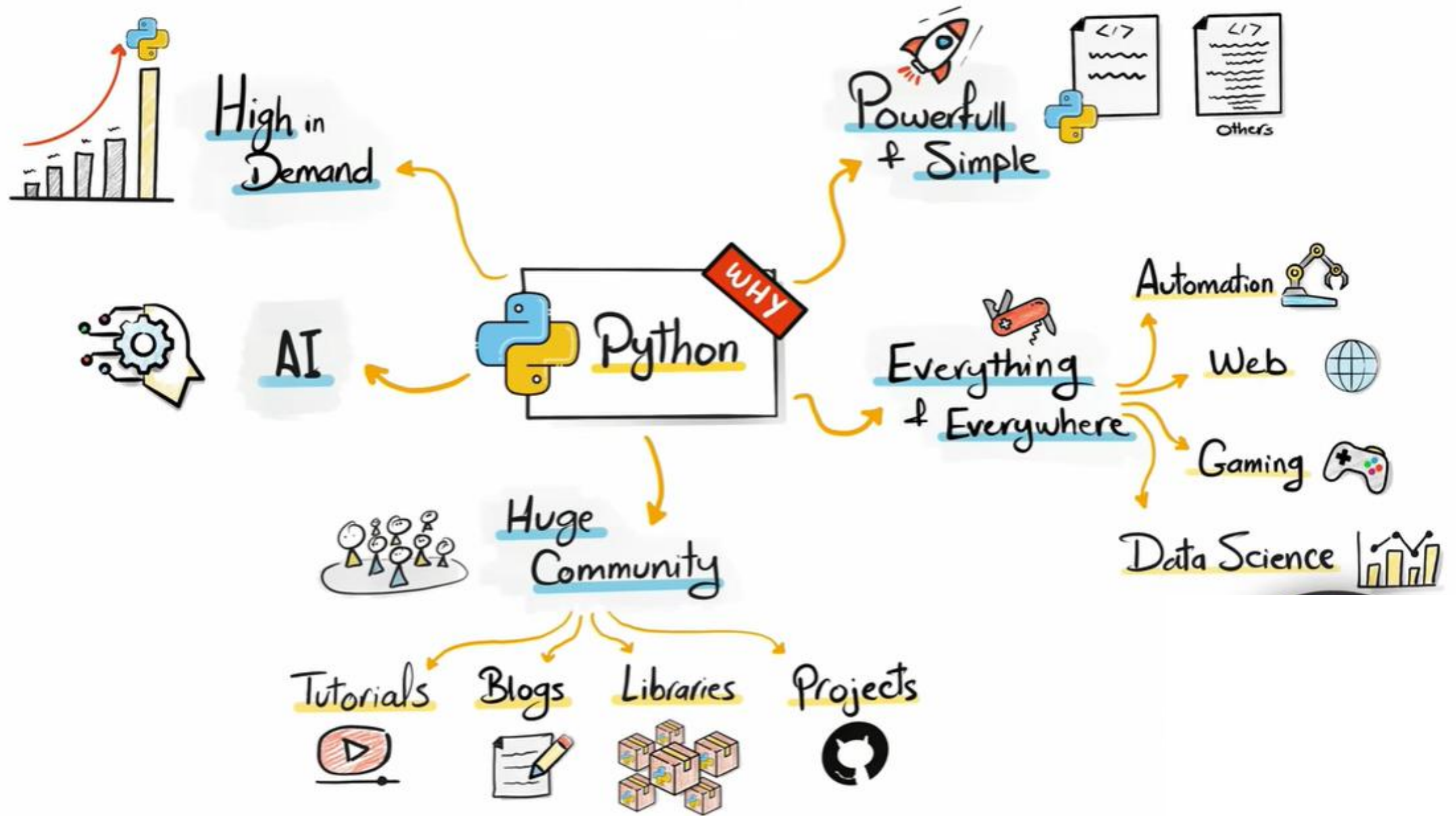
Byte code .pyc

```
3      0 LOAD_CONST          1 (5)
      2 STORE_FAST          0 (a)

4      4 LOAD_CONST          2 (3)
      6 STORE_FAST          1 (b)

5      8 LOAD_FAST           0 (a)
     10 LOAD_FAST           1 (b)
     12 BINARY_ADD
     14 STORE_FAST          2 (c)

6     16 LOAD_GLOBAL          0 (print)
     18 LOAD_FAST            2 (c)
     20 CALL_FUNCTION        1
     22 POP_TOP
     24 LOAD_CONST            0 (None)
```



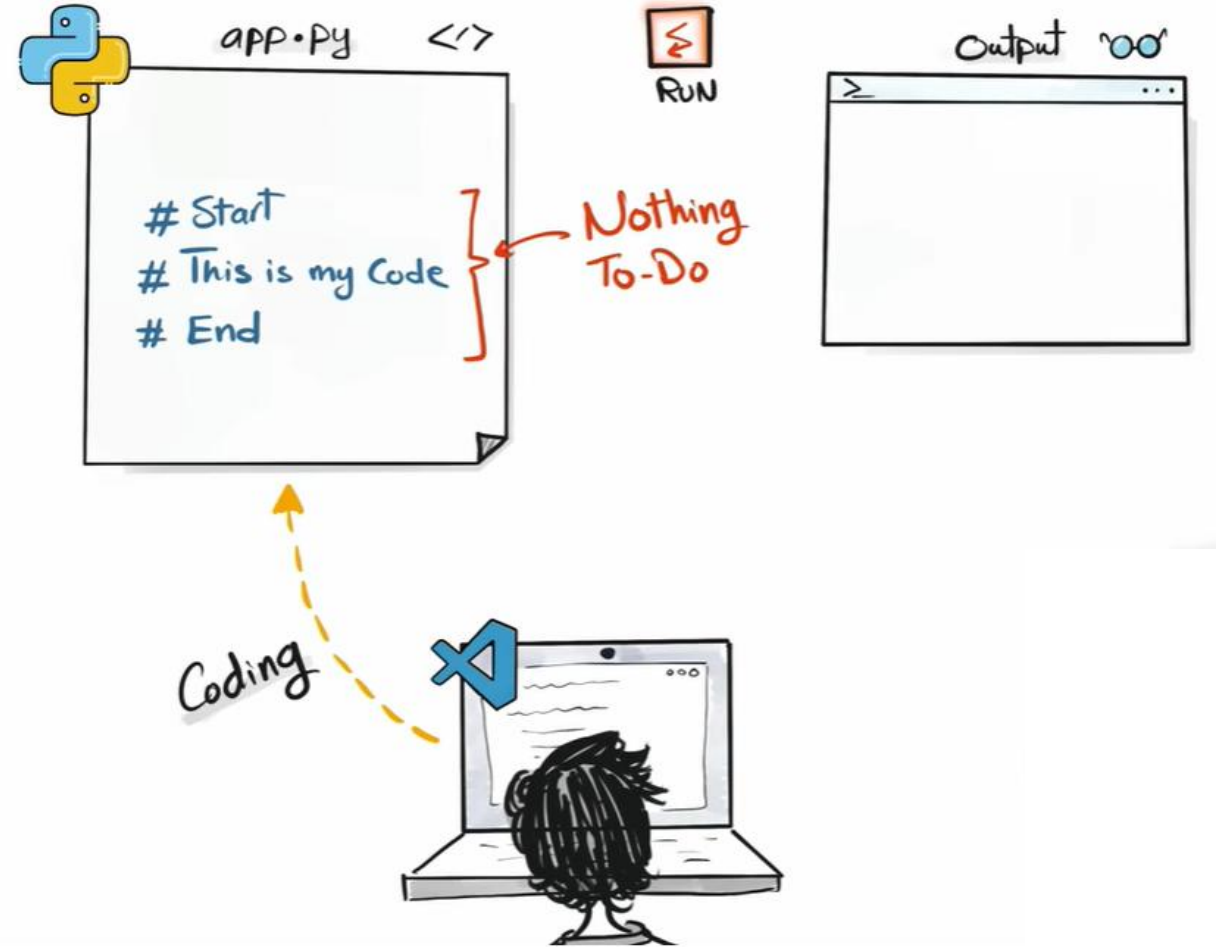


What is a **Comment** ?

A comment is a **note** in your code.
Python just **skips** it!

Why **Comments** are Powerful?

- ✓ Understandable
- ✓ Readable
- ✓ Professional



```
import pandas as pd

df = pd.read_csv("sales.csv")
df = df[df["status"] == "completed"]
df["date"] = pd.to_datetime(df["date"])
df = df[df["date"].dt.month == 4]
df = df[df["region"] == "EMEA"]
df["revenue"] = df["quantity"] * df["unit_price"]
total = df["revenue"].sum()
print(total)
```

```
import pandas as pd

# Load the sales data
df = pd.read_csv("sales.csv")

# Filter only completed sales from April in the EMEA region
df = df[df["status"] == "completed"]
df["date"] = pd.to_datetime(df["date"])
df = df[df["date"].dt.month == 4]
df = df[df["region"] == "EMEA"]

# Compute revenue per row
df["revenue"] = df["quantity"] * df["unit_price"]

# Calculate total revenue and display it
total = df["revenue"].sum()
print(total)
```

Comments

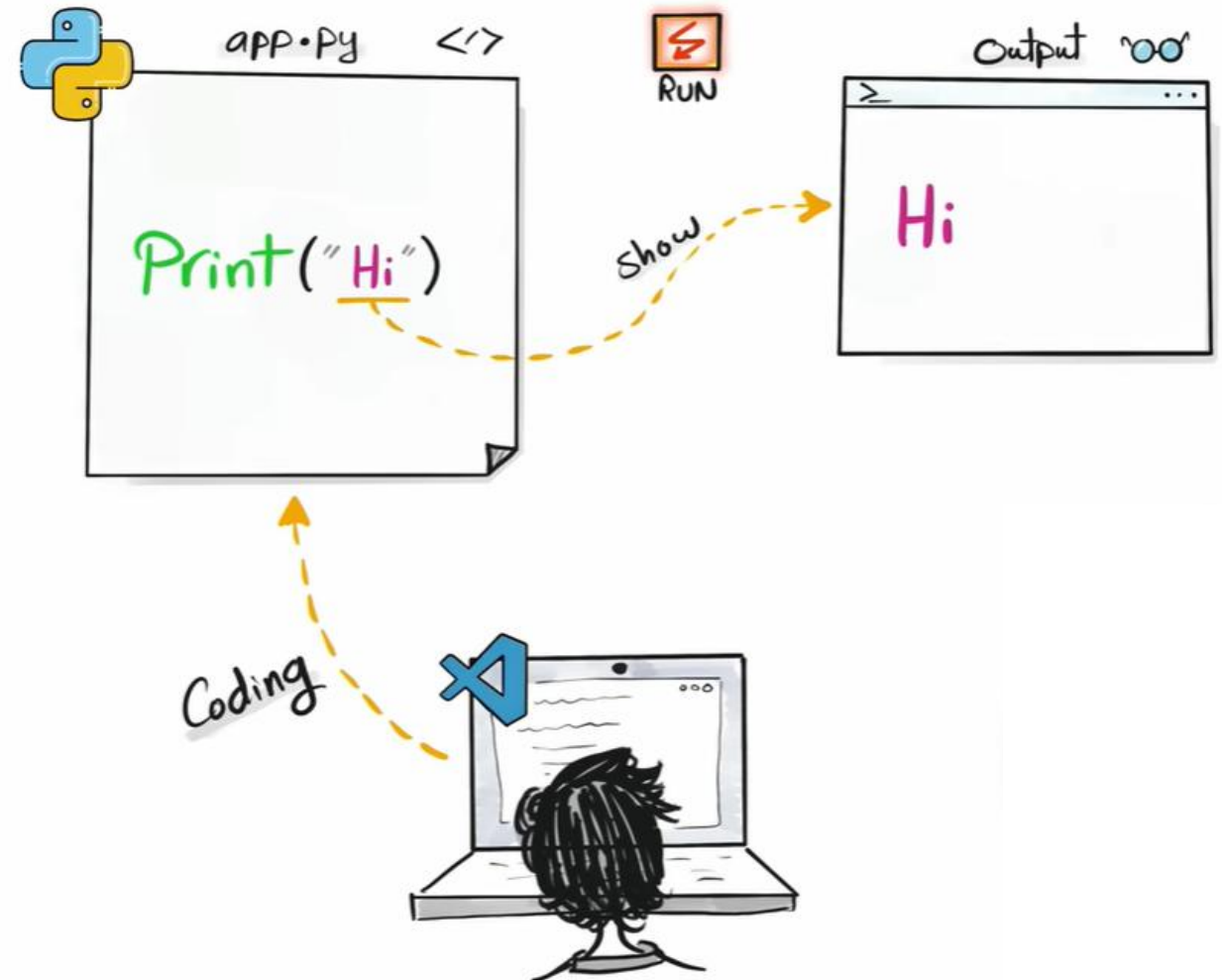


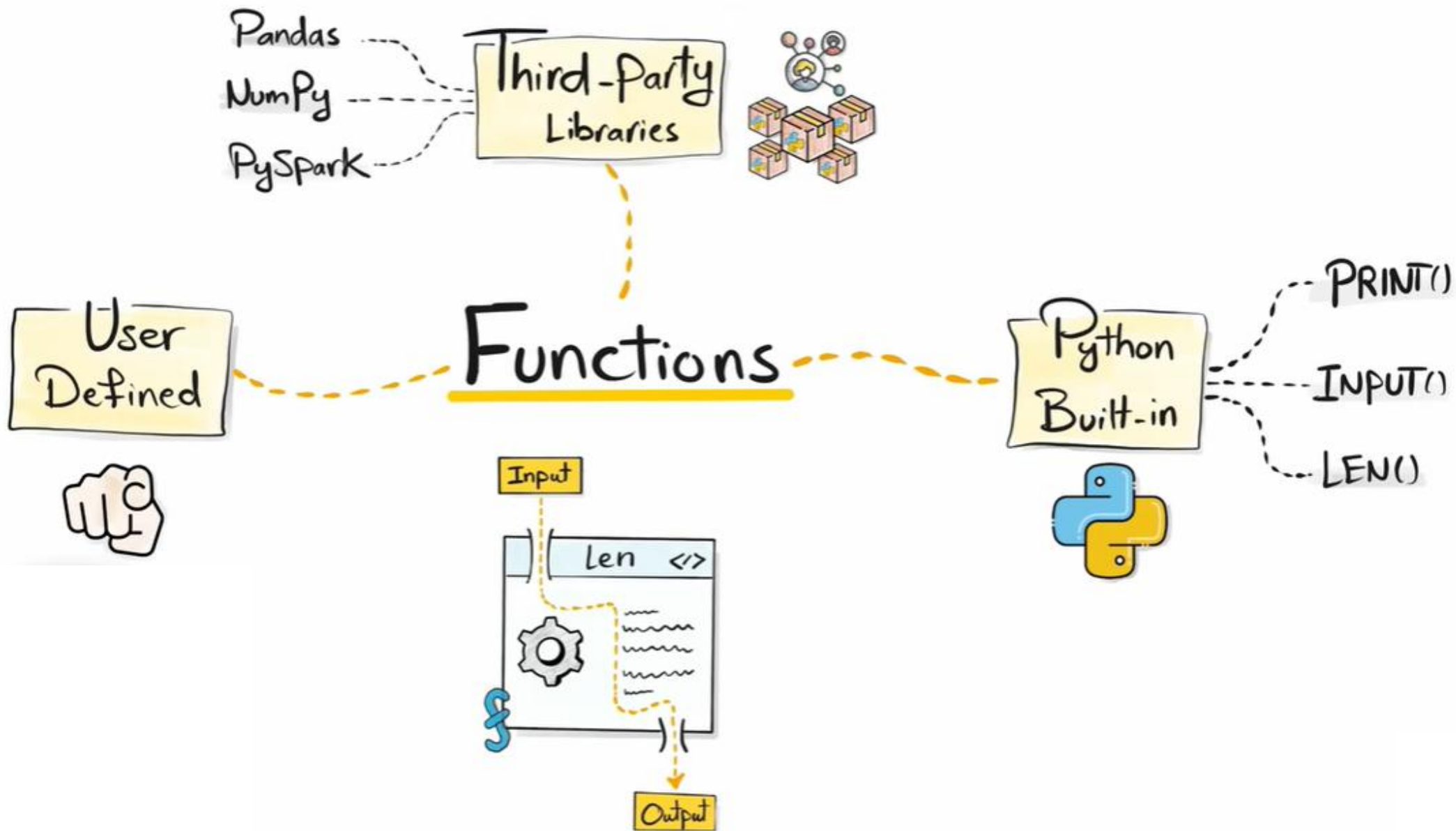
- Comments are notes in code
- Python skips comments
- Comments make code:
Understandable, readable, professional



What is **PRINT()** ?

a built-in Python function that displays messages on the output screen to communicate with users.





Special Characters

" \" ' \n \t "

Normal Characters

A B Z
5 9 0
@ ?

Escape Sequences

\" Double Quote

' Single Quote

\\ Backslash

\n New Line

\t Tab

\b Backspace

Normal Characters "Abe@30n.com"

Normal Characters "Abe@30\n.com"
Break Sequence
Special (NewLine)

```
print("Hi \"Python\"")
```

Start Content End



Python Challenge

- Use PRINT() to recreate this exact output
- You are allowed to use only one PRINT()

Your Learning Path:

- - Python Basics
- - Data Engineering
- - AI

PRINT()

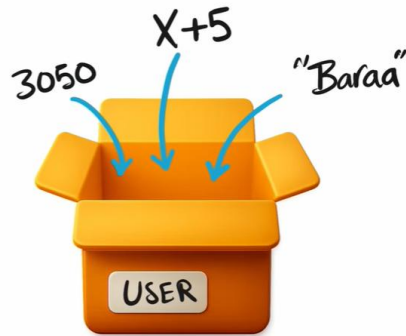


- Built-in Python Function
- Display message in output for users
- Use Cases:
Communicate, Show Results
Debug, Test



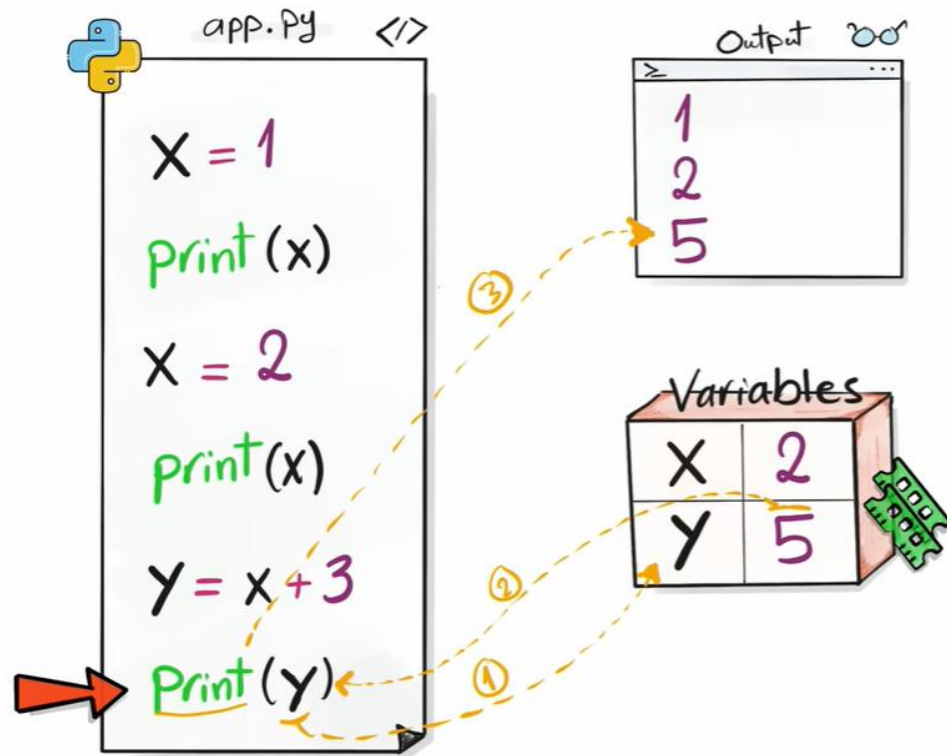
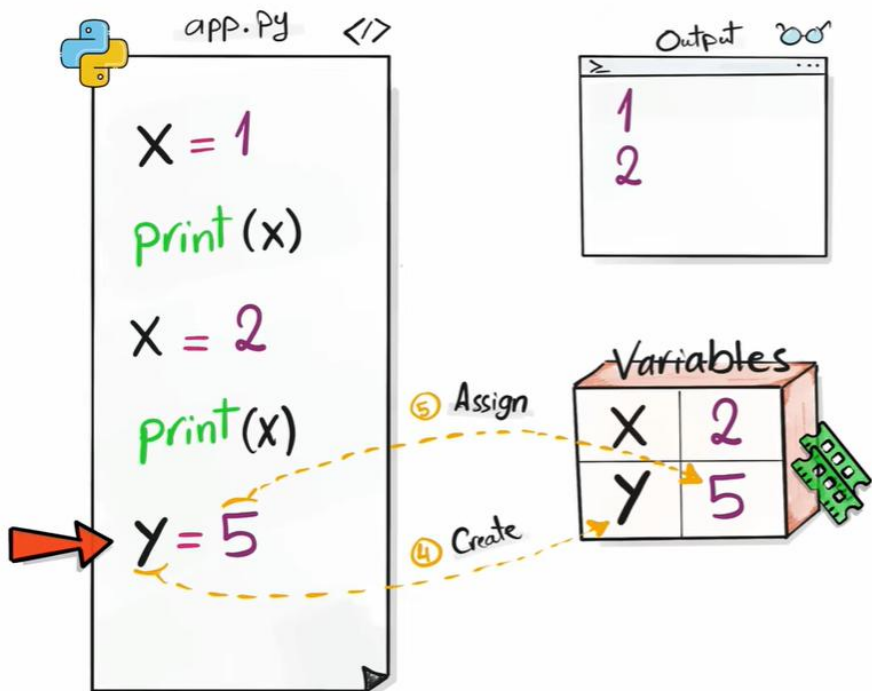
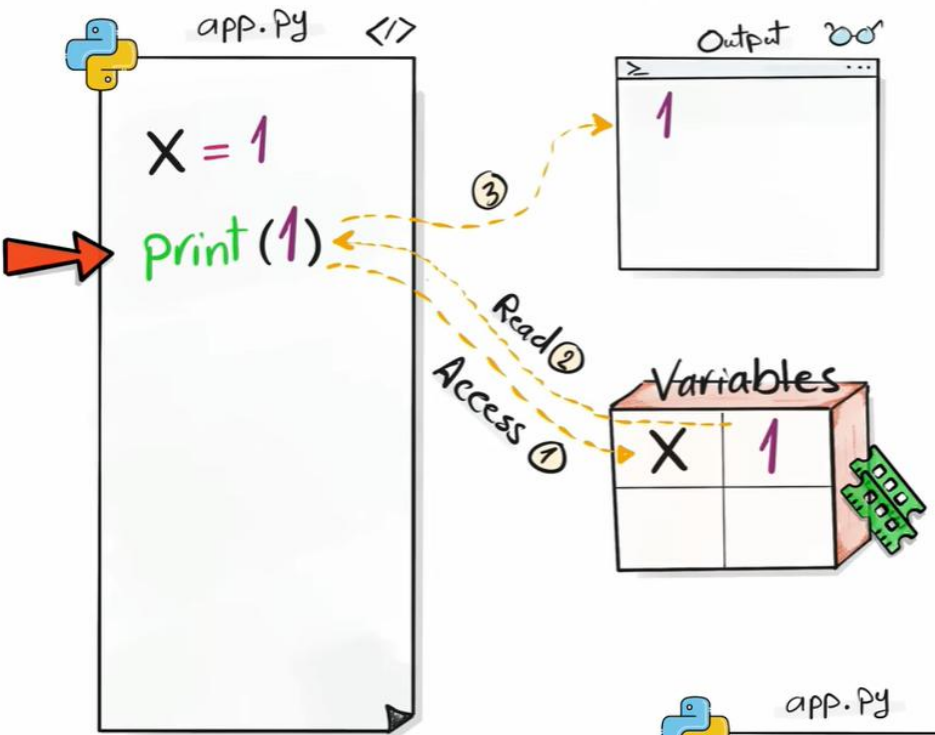
What is a Variable?

A variable is a name you create to store a value, so you can use it later in your code



app.py </>



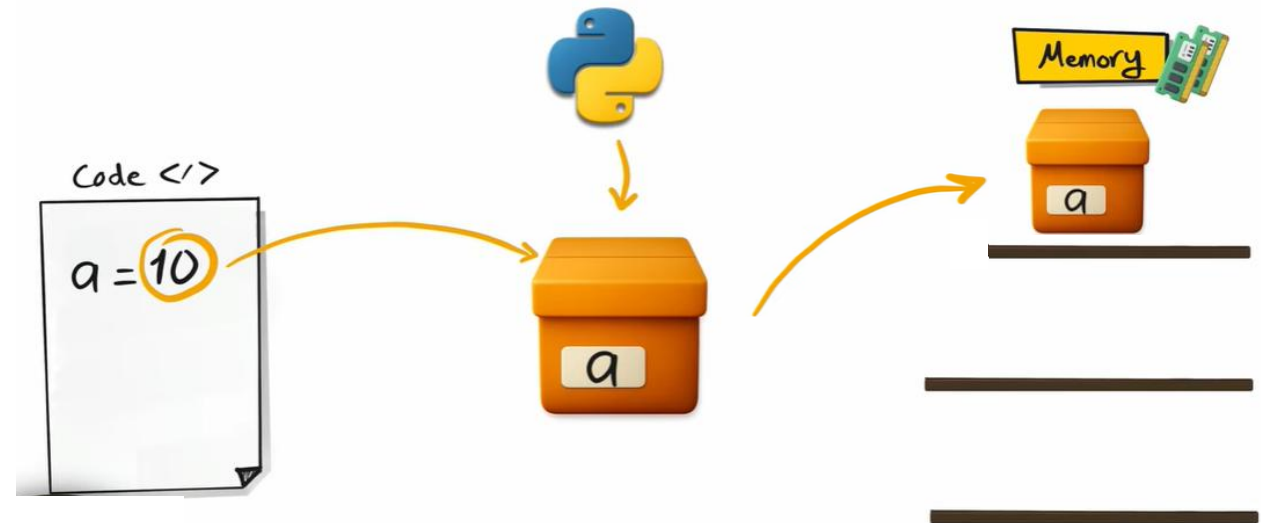
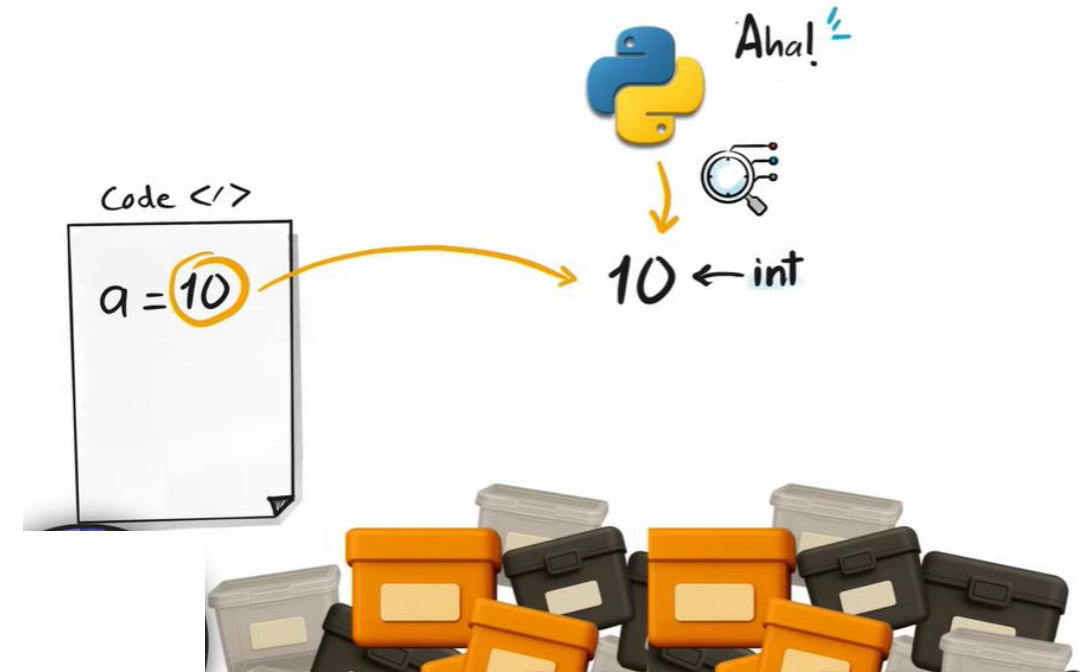
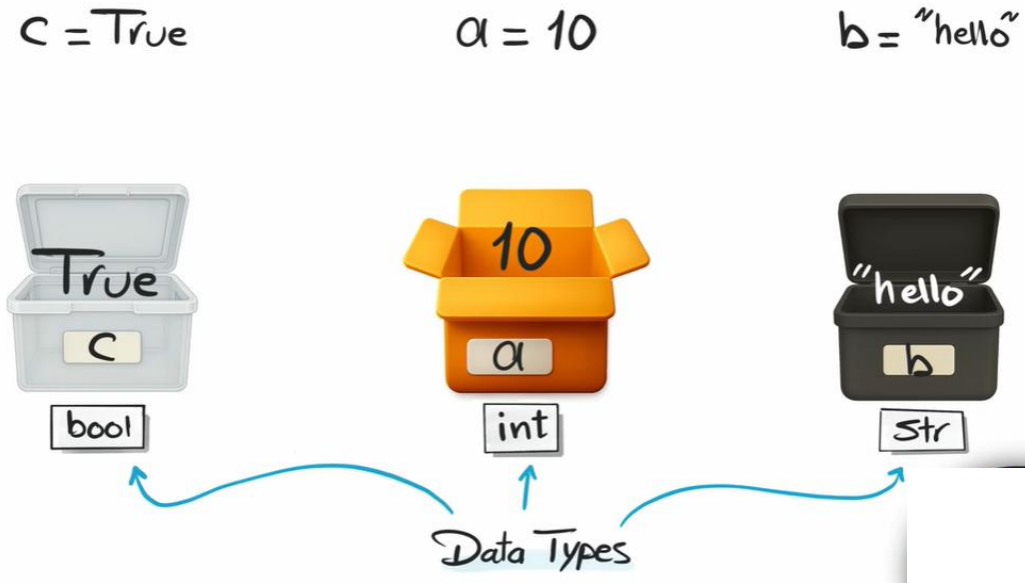


VARIABLES

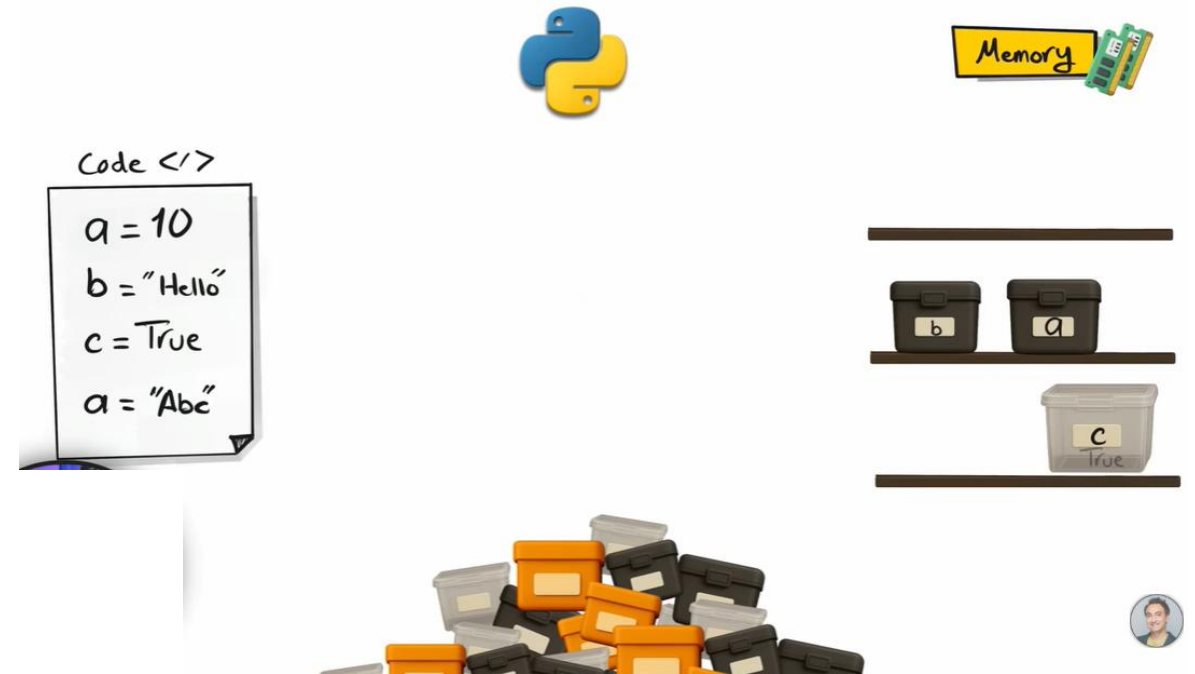
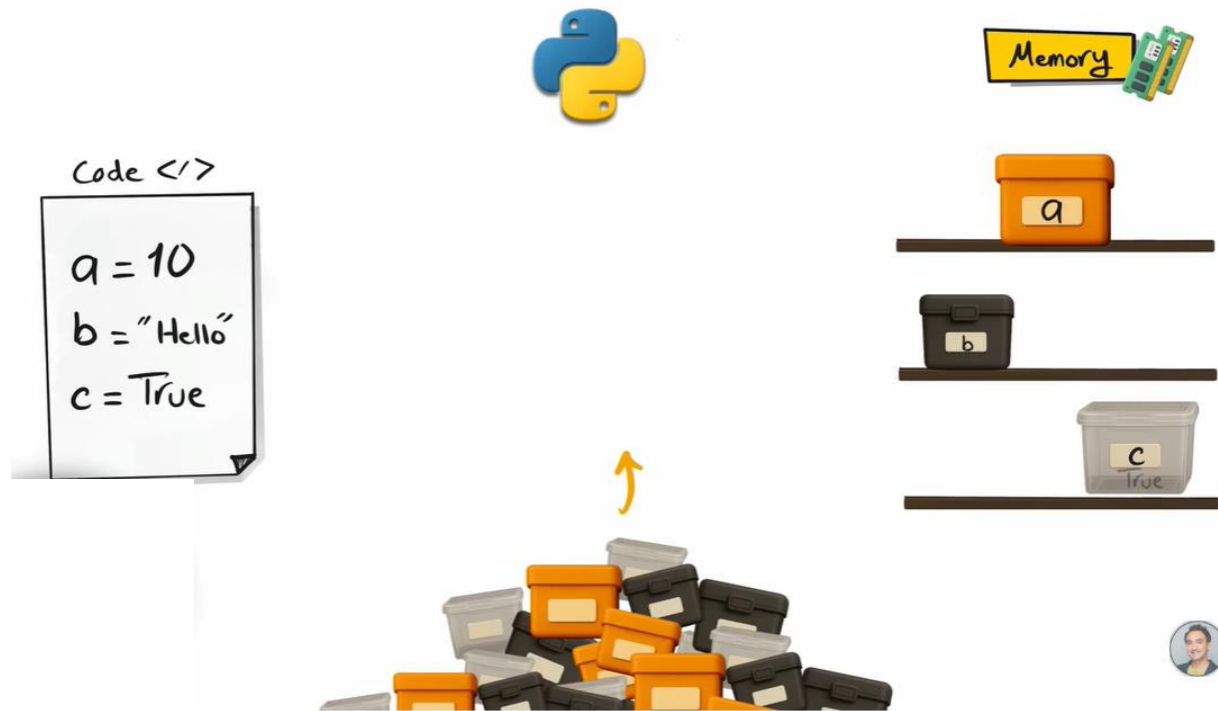


- Make programs dynamic
- Name to store a value
- Stored in memory
- Reusable anytime
- Updatable anytime

Data Types



Data Types



Data Types



Code </>

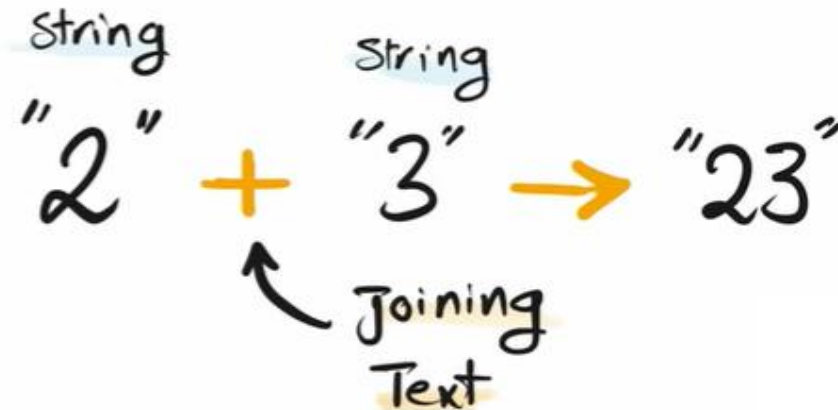
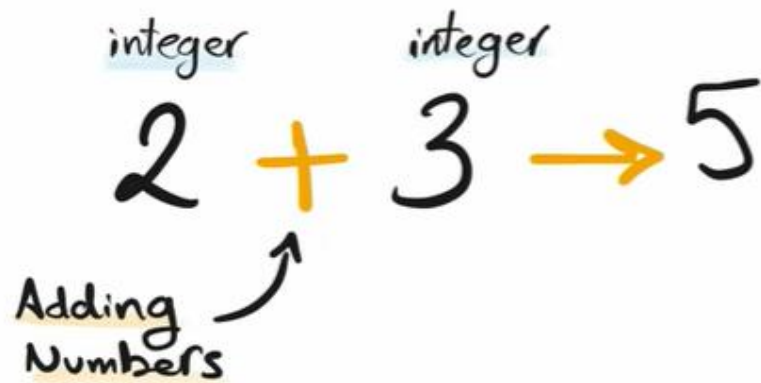
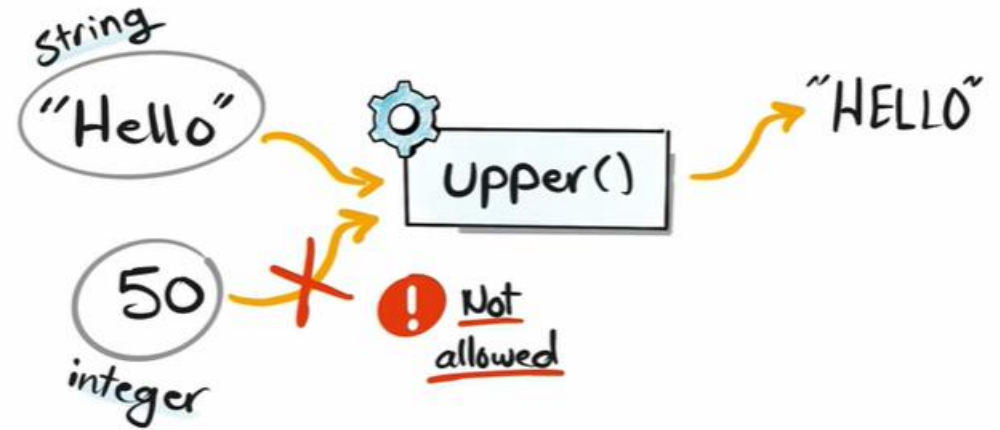
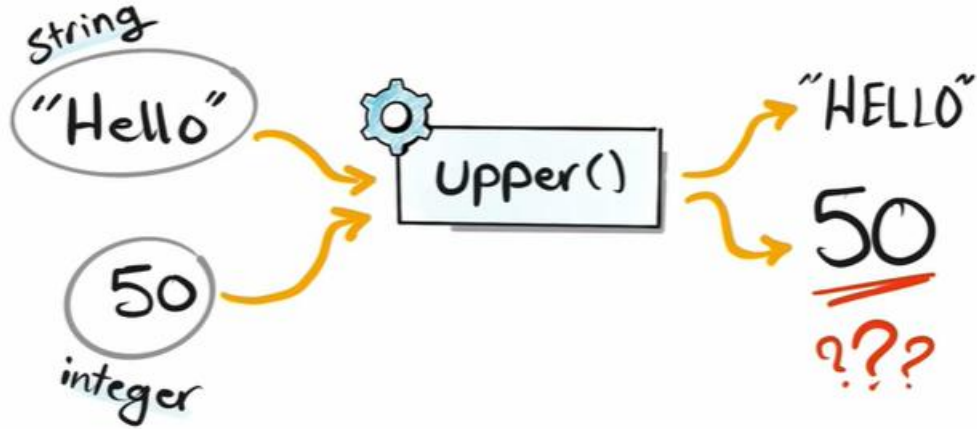
```
a = 10  
b = "Hello"  
c = True  
a = "Abc"
```

- ★ Python Automatically detect Data Types
- ★ Dynamic : Data Types Can change Any Time!

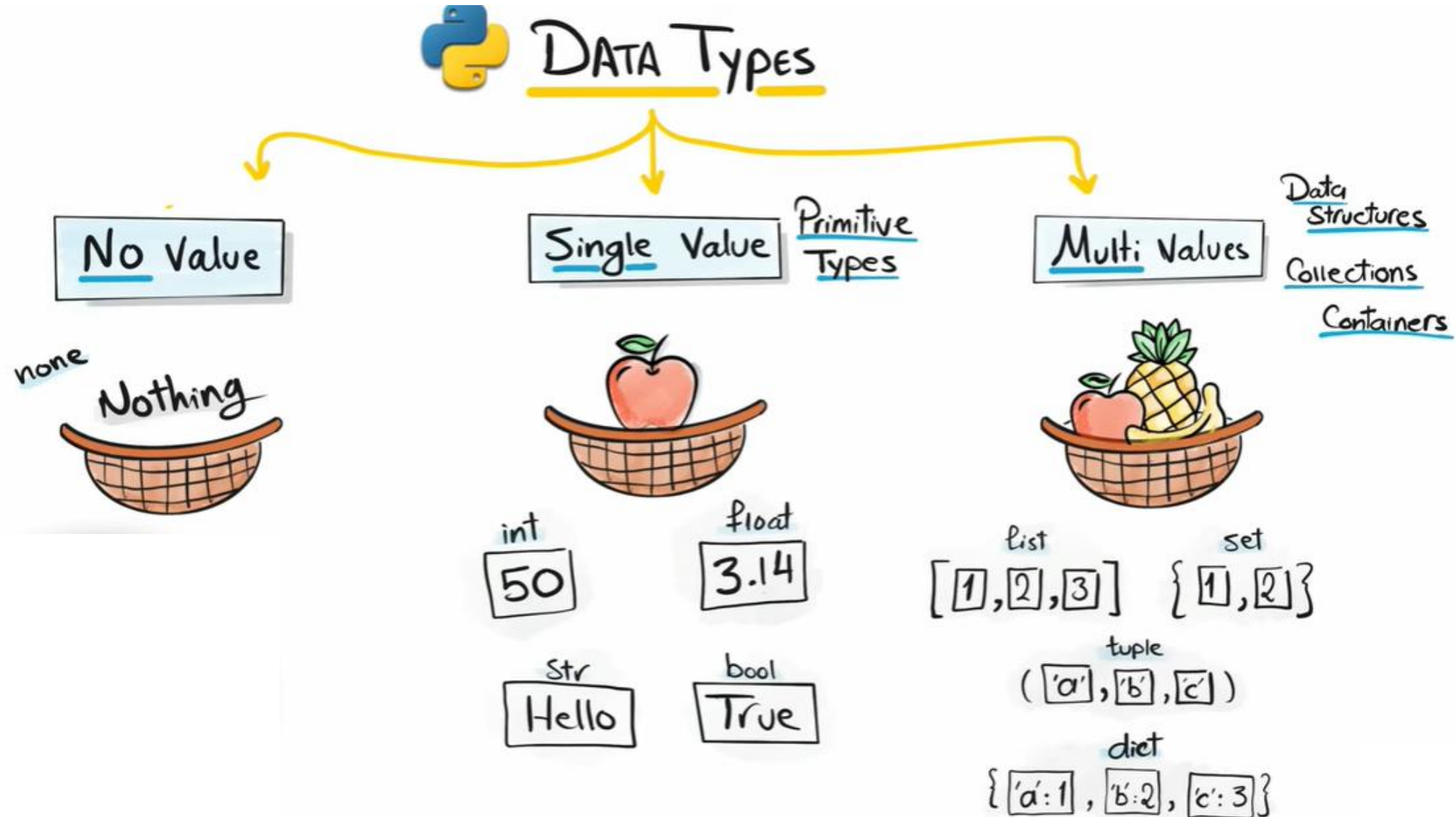
Data
in Different
Types & size



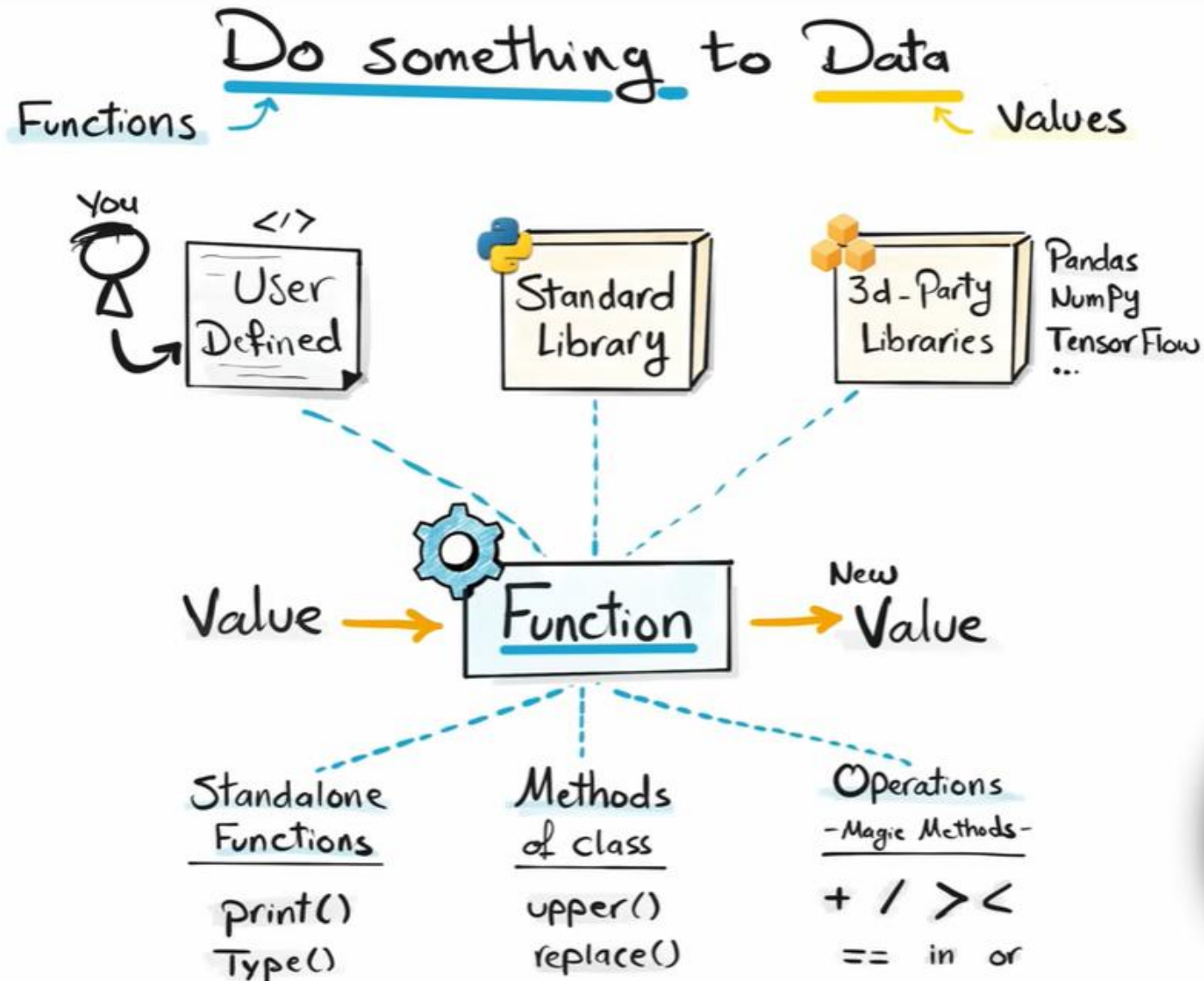
Why Data Types?



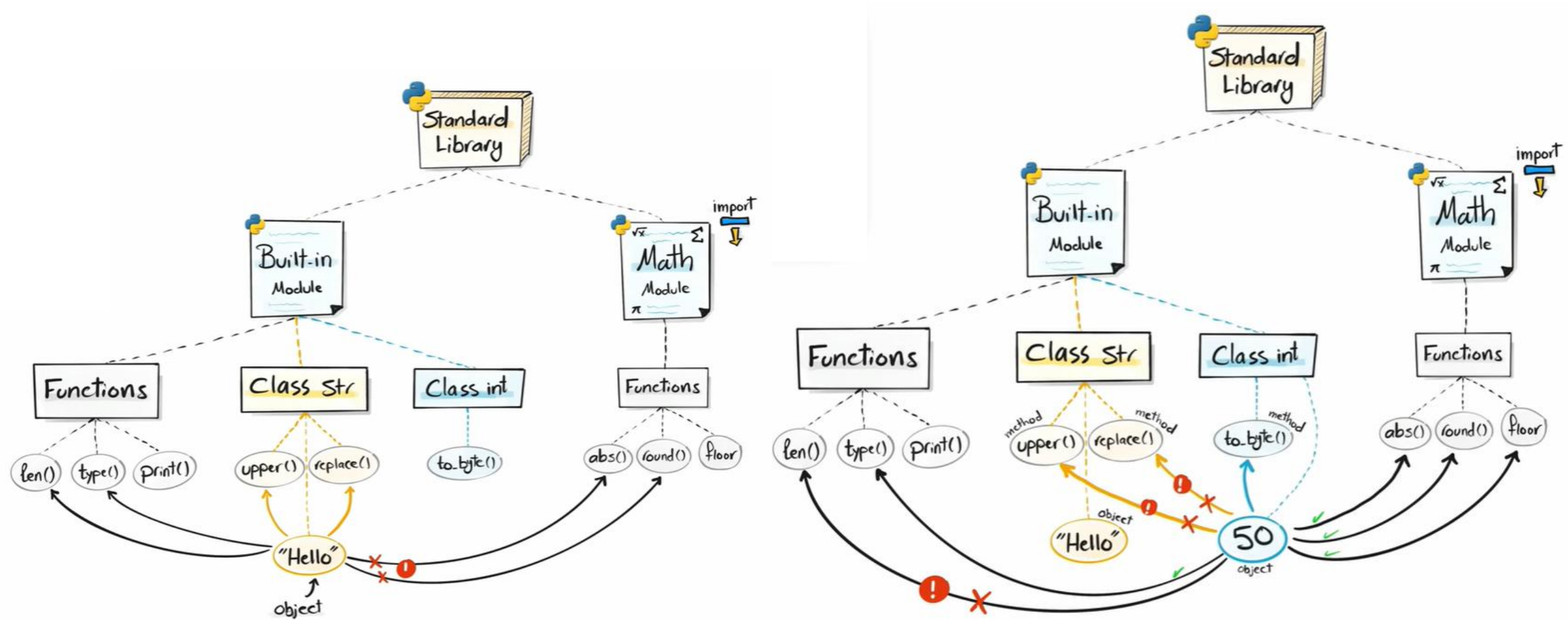
Data Types: Categories



Data Types and Functions



Data Types and Functions



Functions

Independent block of code

Syntax

```
function_name(value)
```

Examples

```
print('hello')  
type(50)
```

Methods

Functions belong to
objects/classes

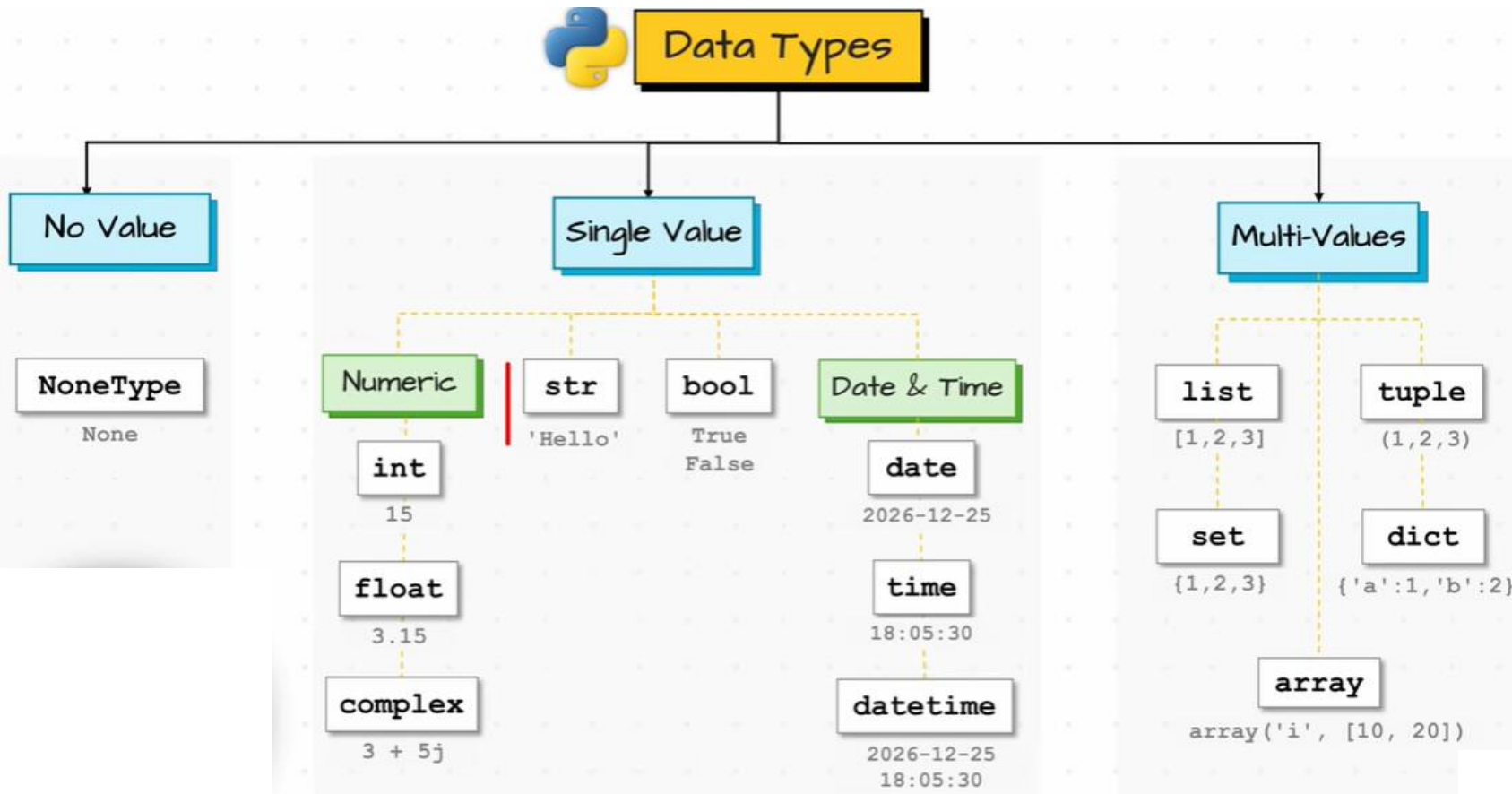
Syntax

```
value.method_name()
```

Examples

```
'hello'.upper()  
50.bit_length()
```

Data Types: Summary



DATA TYPES



- Each value has a data type
This helps Python decide how to handle the value
- It automatically detects data types
- Dynamic: Data Types can change anytime
- 3 Categories
 - No Value: `NoneType`
 - Single Value: `int`, `str`, `bool`, `float`, `complex`
 - Multiple Values: `set`, `tuple`, `dict`, `list`
- Values are objects of a data type class